

```
1 var el = document.getElementById('miDiv');
2 el.setAttribute('data-secreto', 'Mucho...');
```

Eventos

Una de las mayores barreras a la hora de introducir el dinamismo en una web es la gestión de los eventos, ya que estos son asíncronos por naturaleza, por lo que no puedes saber de antemano. Por ejemplo, cuántos segundos tardará un usuario en pinchar sobre un botón... lo que sumado a un funcionamiento peculiar con propagaciones, hace que el dominio de los eventos sea una tarea complicada.



Sin embargo, todo gran esfuerzo tiene una gran recompensa y el dominio de estos eventos nos abrirá las puertas a la *programación dirigida a eventos*, y posteriormente a *Node.js*, si decidimos dar el gran paso hacia el *Back-End* o el *Full Stack*.

Funcionamiento

Básicamente podemos seleccionar un elemento de HTML, y suscribir uno de los posibles eventos de los que dispone:

- *Eventos de ratón*
- *Eventos de teclado*
- Y *muchos más...*

Una vez hemos definido estos detalles, en dónde escuchamos y qué esperamos, solo queda definir que haremos cuando esto ocurra.

Lógicamente al tratarse de asincronía, debemos definir una función, ya sea en línea o reutilizable para gestionarlo todo.

Si necesitamos información adicional sobre el evento sucedido, el propio sistema mandará *un objeto con detalles clave sobre el evento*, como argumento a la hora de ejecutar nuestro callback.

Utilizando eventos

Existen dos formas básicas de añadir eventos a nuestra aplicación. Una es por medio de html con atributos como *onclick*, y la otra desde el JavaScript, haciendo uso de métodos como *.addEventListener()*.

Cuál es mejor o cuál es peor... realmente varia en función de las circunstancias del código y su programador. Yo creo firmemente, que es mucho mejor separar JavaScript, HTML y CSS.

Así evitamos el *antipatrón* de mezclar JavaScript en el HTML. Por otra parte, si encapsulas tu código en una función anónima autoejecutada, no podrás acceder a las funciones directamente desde el HTML...

Así que por estos motivos principales, recomendamos encarecidamente el uso de *.addEventListener()*.

Añadir un evento

Veamos como funciona con *onclick*.

```
1 <body onclick="cambiarFondo()"></body>

1 function cambiarFondo() {
2     // color = 'rgb(0-255,0-255,0-255)'
3     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
4     color += Math.floor((Math.random() * 255)) + ',';
5     color += Math.floor((Math.random() * 255)) + ')';
6     document.body.style.backgroundColor= color;
7     console.info("Nuevo color:", color);
8 }
```

Veamos como funciona con *addEventListener()*.

```
1 document.body.addEventListener('click', function (e) {
2     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
3     color += Math.floor((Math.random() * 255)) + ',';
4     color += Math.floor((Math.random() * 255)) + ')';
5     document.body.style.backgroundColor= color;
6     console.info("Nuevo color:", color);
7 });
```

Eliminar un evento

Los eventos ocupan memoria. Es un factor a tener en cuenta, especialmente, si queremos hacer una web con un buen soporte para smartphones. Si un evento deja de tener sentido, sencillamente puedes eliminarlo con *.removeEventListener()*.

```
1 function cambiarColor (){
2     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
3     color += Math.floor((Math.random() * 255)) + ',';
4     color += Math.floor((Math.random() * 255)) + ')';
5     document.body.style.backgroundColor= color;
6     console.info("Nuevo color:", color);
7 }
8
9 document.body.addEventListener('click', cambiarColor);
10 document.body.removeEventListener('click', cambiarColor);
```

Manejadores de eventos

Cuando se dispara un evento, podemos recopilar mucha información útil, si no ignoramos el argumento que nos pasan.

Podemos fácilmente crearnos una función que nos aporte información real sobre el evento y que no ejecute ninguna acción en nuestra aplicación. Este tipo de funciones son muy útiles y deberían estar en tu kit de herramientas para desarrollar webs como un artesano de verdad.

```
1  function manejadorEventos(elEvento) {
2      // Compatibilizar el evento
3      var evento = elEvento || window.event;
4
5      // Imprimir detalles
6      console.log("-----")
7      console.log("Type: "+evento.type); // Tipo
8      console.log("Bubbles: "+evento.bubbles);
9      console.log("Cancelable: "+evento.cancelable);
10     console.log("CurrentTarget: ", evento.currentTarget);
11     console.log("DefaultPrevented: "+evento.defaultPrevented);
12     console.log("EventPhase: "+evento.eventPhase);
13     console.log("Target: ", evento.target);
14     console.log("TimeStamp: "+evento.timeStamp);
15     console.log("IsTrusted: "+evento.isTrusted); // true - Usuario
16     console.log("=====")
17 }
18
19 // Añadimos Listener
20 document.addEventListener('click', function(evento){
21     manejadorEventos(evento);
22     // Más código
23 });
```

Usos Avanzados

Lanzar un evento manualmente con `.dispatchEvent()`

```

1 document.body.addEventListener('click', function (e) {
2     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
3     color += Math.floor((Math.random() * 255)) + ',';
4     color += Math.floor((Math.random() * 255)) + ')';
5     document.body.style.backgroundColor= color;
6     console.info("Nuevo color:", color);
7 });
8
9 var lanzadorEventos = new Event('click');
10 document.body.dispatchEvent(lanzadorEventos);

```

Compatibilidad con Internet Explorer <= IE8

Si tienes la mala suerte de tener que dar soporte a versiones obsoletas de Internet Explorer, además de todo el sufrimiento acumulado, deberás añadirte una carga extra... y es que por aquel entonces Internet Explorer, no era compatible con `addEventListener()`, ni con `removeEventListener()`, así que necesitamos utilizar `.attachEvent()` y `.detachEvent()`.

`.attachEvent()`

```

1 document.attachEvent('onclick', function (e) {
2     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
3     color += Math.floor((Math.random() * 255)) + ',';
4     color += Math.floor((Math.random() * 255)) + ')';
5     document.body.style.backgroundColor= color;
6     console.info("Nuevo color:", color);
7 });

```

`.detachEvent()`

```

1 function cambiarColor (){
2     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
3     color += Math.floor((Math.random() * 255)) + ',';
4     color += Math.floor((Math.random() * 255)) + ')';
5     document.body.style.backgroundColor= color;
6     console.info("Nuevo color:", color);
7 }
8
9 document.body.attachEvent('onclick', cambiarColor);
10 document.body.detachEvent('onclick', cambiarColor);

```

Dado este panorama de tener que usar dos métodos diferentes, cuando además tienen una estructura similar, pero no idéntica... tenemos un reto entre manos que merece una solución duradera.

Aunque descartamos enseñar el uso de *patrones*, sería injusto por nuestra parte no arrojar al menos un poco de luz sobre el asunto, así que nos gustaría mostrarte un par de trucos de “artesano veterano”.

Vamos a crear un objeto (literal), donde guardaremos todo lo necesario para manejar eventos, subiendo así el nivel de abstracción de vuestra aplicación.

Ya no usaremos `.attachEvent()` o `.addEventListener()`. Ahora utilizaremos `.agregar()`, que internamente llamará a `.attachEvent()` o `.addEventListener()`, en función del navegador en el que nos encontremos.

Cambiar de navegador en medio de la ejecución del script parece altamente improbable. No tiene mucho sentido comprobar cada vez que agreguemos un evento, si usaremos `.attachEvent()` o `.addEventListener()`.

Por eso los patrones de diseño pueden ayudarnos a solucionar cosas así. En este caso usaremos *Init-time branching*, que es una variante de *Lazy initialization*.

Básicamente crearemos un objeto con los métodos *agregar* y *quitar* vacíos. Al ejecutarse el código por primera vez, comprobaremos que navegador usamos y en función de ello sobrescribirá *agregar* y *quitar*, llamando por debajo a los métodos correspondientes.

Es un poco más de trabajo de lo esperado, pero así es mucho mas sencillo gestionar nuestra aplicación, ya que nuestros métodos se adaptarán y no así nuestro código.

El valor añadido de usar este patrón esta, en que en un solo punto tomo una decisión que se extenderá por toda nuestra aplicación.

```

1  var eventos = {
2      agregar: null,
3      quitar: null,
4      manejador: function(evento) {
5          console.group("Manejador de Eventos");
6          console.log("-----");
7          console.log("Type: " + evento.type); // Tipo
8          console.log("Bubbles: " + evento.bubbles); // sube por el DOM
9          console.log("Cancelable: " + evento.cancelable);
10         console.log("CurrentTarget: ", evento.currentTarget);
11         console.log("DefaultPrevented: " + evento.defaultPrevented);
12         console.log("EventPhase: " + evento.eventPhase);
13         console.log("Target: ", evento.target);
14         console.log("TimeStamp: " + evento.timeStamp);
15         console.log("IsTrusted: " + evento.isTrusted); // true - Usuario
16         console.log("=====");
17         console.groupEnd();
18     }
19 }
```

```

20
21 // Init-time branching (Patrón)
22 if (typeof window.addEventListener === 'function') {
23     eventos.agregar = function(el, type, fn) {
24         el.addEventListener(type, fn, false);
25     };
26     eventos.quitar = function(el, type, fn) {
27         el.removeEventListener(type, fn, false);
28     };
29 } else { // Soporte para <= IE8
30     eventos.agregar = function(el, type, fn) {
31         el.attachEvent('on' + type, fn);
32     };
33     eventos.quitar = function(el, type, fn) {
34         el.detachEvent('on' + type, fn);
35     };
36 }
37
38 eventos.agregar(document.body, 'click', function (e) {
39     var color = 'rgb(' + Math.floor((Math.random() * 255))+ ',';
40     color += Math.floor((Math.random() * 255)) + ',';
41     color += Math.floor((Math.random() * 255)) + ')';
42     document.body.style.backgroundColor= color;
43     console.info("Nuevo color:", color);
44 })

```

Delegación de Eventos

Una de las técnicas más útiles para ahorrar memoria, es utilizar la *delegación de eventos*, es decir, en vez de poner un evento por cada uno de los elementos que compone una estructura de datos, es preferible hacerlo únicamente, sobre el elemento padre común a todos ellos y simplemente filtrar cuál de los hijos fue disparado.

Modo Clásico (sin delegación)

```

1 <ul id="miNav">
2     <li><a href="#nosotros">¿Quienes Somos?</a></li>
3     <li><a href="#objetivos">Los objetivos</a></li>
4     <li><a href="#equipo">Nuestro Equipo</a></li>
5     <li><a href="#detalles">Más detalles</a></li>
6     <li><a href="#contacta">Contactanos</a></li>
7 </ul>

```

```

1  var miNav = document.getElementById("miNav");
2  var miNavLinks = miNav.getElementsByTagName("a");
3
4  for (var i = 0; i < miNavLinks.length; i++) {
5      miNavLinks[i].onclick = function(){
6          console.info(this.innerHTML);
7      }
8  };

```

Modo Delegando

```

1  var miNav = document.getElementById("miNav");
2  miNav.onclick = function(evento){
3
4      var evento = evento || window.event;
5      var elemento = evento.target || evento.srcElement;
6      console.info(elemento.innerHTML);
7  };

```

Creación de Eventos Personalizados

Otra estrategia más avanzada, es hacer nuestro código más modular e interconectado por eventos, aunque esto es más típico de Node.js. En el lado del cliente también podemos hacer uso de ello.



Como verás incluso podemos disparar eventos, lo que amplía mucho las posibilidades para jugar con páginas web existentes desde la consola... *¡Yo no digo nada...!*

```

1  var evento = new Event('miEventoInventado');
2
3  document.body.addEventListener('miEventoInventado', function (e) {
4      console.info(e); // {isTrusted: false}
5  });
6
7  document.body.dispatchEvent(evento);

```

Propagación (Capturing y Bubbling)

Uno de los puntos de fricción más altos a la hora de trabajar con eventos, es entender como funciona la propagación que JavaScript realiza.

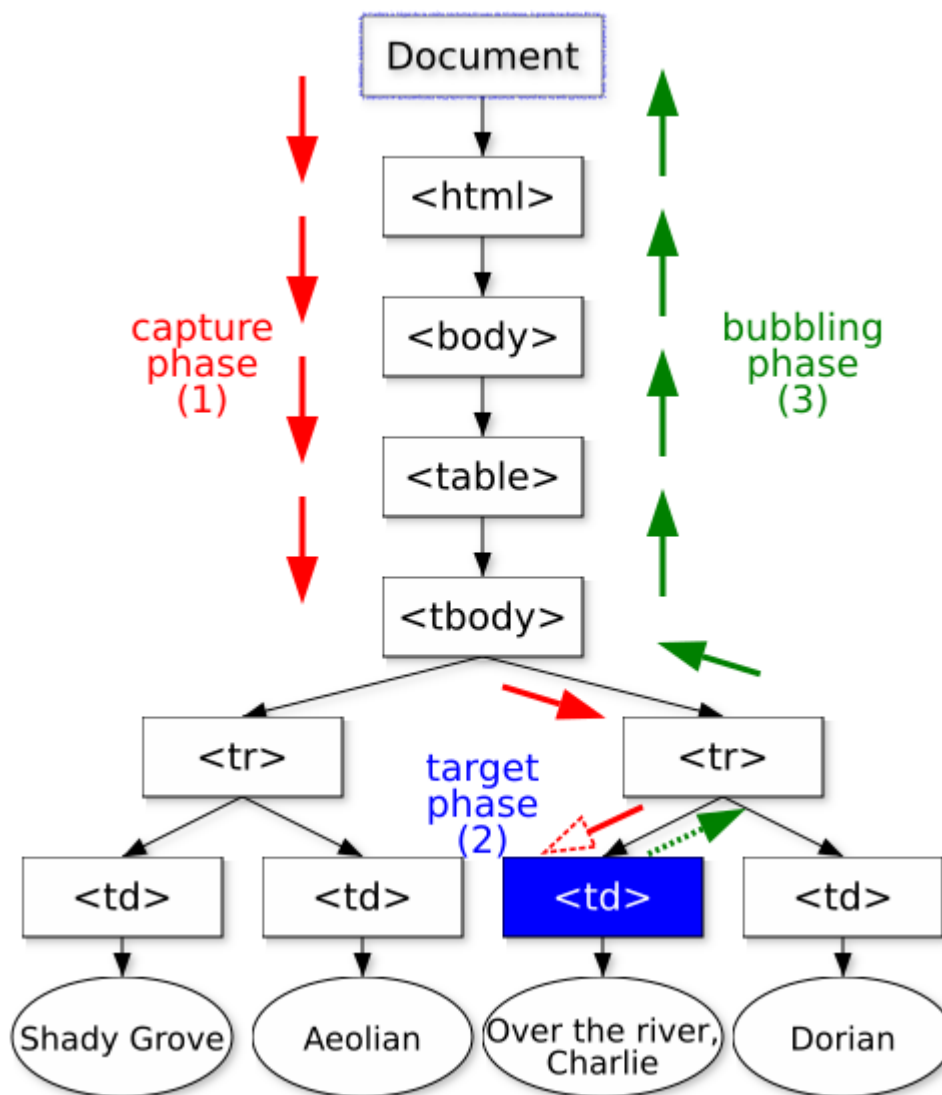


Imagen obtenida de W3C

La mejor manera de entenderlo es con un *ejemplo*.

Si ejecutas el evento verás que existe un pequeño conflicto de intereses en el HTML.

```

1 <div id="parent">
2   <div id="children">
3     Click
4   </div>
5 </div>

```

Tanto *parent* como *children* tienen eventos suscritos. El problema básico, que nos encontramos es que *children* está envuelto por *parent*, lo que hace que pinchemos, donde pinchemos siempre se disparan ambos.

Esto se debe a la propagación, que funciona en tres fases:

Capturing

Recorre todo el documento desde *document* hasta el elemento en sí.

Target

Es cuando se llega al elementos en cuestión.

Bubbling

Realiza el recorrido de vuelta hacía arriba desde el elemento en sí.

Podemos interrumpir este proceso de propagación si hacemos uso de [*stopPropagation\(\)*](#).

Como has podido ver, el ejemplo en sí que hemos utilizado aquí, fuerza un poco la situación, ya que existe mucho código duplicado y gestiona los eventos de una manera incorrecta.

Otro método interesante, que no debemos perder de vista es [*.preventDefault*](#). Evita el comportamiento por defecto, por ejemplo si queremos que un link no actúe como tal.

Al contrario de lo que pueda parecer ahora mismo, la propagación es uno de los mejores aliados a la hora de gestionar nuestros eventos, como vimos anteriormente con la *delegación de eventos*.